IF25-22014 PENGEMBANGAN APLIKASI WEB

Pertemuan 7
Kerangka Kerja React JS

Membangun antarmuka dari komponen React, dijalankan dengan Next.js

Tim Pengajar

Tahun 2026. Rujukan: Dokumentasi resmi React (`react.dev`) dan Next.js (`nextjs.org/docs`); Algifari dkk., ITERA Press, 2024, bab React JS.

ITERA

DIKTISAINTEK BERDAMPAK

Program Studi Teknik Informatika
Institut Teknologi Sumatera

PETA PERTEMUAN

Perjalanan kalian hari ini

ITERA

Fase	Menit	Yang kalian lakukan	Pertanyaan wajar	Bila tersendat
Pembuka	0–10	Menjawab tiga soal minggu lalu dengan jari	Masih ingat yang kemarin?	Catat nomor yang salah, cek di modul minggu lalu
Coba dulu	10–30	Mengerjakan tiga misi berpasangan	Boleh salah?	Tulis dugaan; salah itu bahan belajar
Materi	30–85	Mengikuti 8 bagian: tebak, ikuti bersama, engsel	Sedang di bagian mana?	Lihat penanda di atas judul atau klik Outline
Hands-on	85–120	Mengerjakan tingkat 1 sampai 3 dari `starter/`	Sudah benar belum?	Periksa berpasangan; buka modul bagian yang sama
Papan	120–140	Menunjukkan atau menanggapi hasil teman	Kenapa kode itu jalan?	Ajukan satu pertanyaan mengapa
Exit ticket	140–150	Menjawab dua soal tanpa catatan	Apa yang belum paham?	Tulis di kolom refleksi

Penanda fase dan bagian ada di atas setiap judul slide. Kotak **Ikuti bersama** berarti ketik di laptop kalian sekarang.

Soal dari Pertemuan 6

ITERA

Soal 1. Hasil `[1,2,3].map((n) ⇒ n * 2)`?

- **A** 6
- **B** [1,2,3]
- **C** [2,4,6]
- **D** [3]

Soal 2. fetch mendapat 404. Yang terjadi?

- **A** error dilempar
- **B** res.ok false
- **C** await gagal
- **D** halaman dimuat ulang

Soal 3. `const [a, b] = useState(0)` adalah contoh?

- **A** spread
- **B** destructuring array
- **C** class
- **D** module

KUNCI

1C: map mengubah setiap isi. 2B: periksa res.ok. 3B: dibongkar menurut posisi.

Sepuluh kartu, sepuluh kali salin

ITERA

Halaman KRS kalian dari P5 menampilkan setiap mata kuliah sebagai kartu. Kode kartunya disalin sepuluh kali. Saat desain kartu diubah, sepuluh tempat harus diedit. Lalu total SKS di pojok halaman harus diperbarui manual setiap kali kartu dicentang, dan dua kali lupa.

PIKIRKAN

Bagaimana menulis kartu sekali saja, dan membuat total SKS selalu benar dengan sendirinya?

```
<div class="kartu">
  <h3>Pemweb</h3><p>3 SKS</p>
</div>
<div class="kartu">
  <h3>PAM</h3><p>3 SKS</p>
</div>
<!— ... delapan lagi —>
<p>Total: <span id="t">6</span></p>
```

Misi kalian, 20 menit

ITERA

Berdua atau bertiga, pakai apa saja yang kalian tahu. Berkas awal ada di folder hands-on, `starter/`.

1 Buat proyek

Jalankan `npm install` lalu `npm run dev` di `starter/`. Buka `localhost:3000`. Berkas mana yang menjadi halaman ini?

2 Halaman baru

Tanpa contoh, coba buat halaman yang muncul di `localhost:3000/tentang`.

3 Satu kartu

Di `components/KartuMatkul.js`, buat kartu yang menerima nama dan SKS, lalu pakai tiga kali di halaman utama.

Boleh

Bertanya ke teman, mencoba di browser, salah.

Belum dulu

Mencari di internet atau bertanya ke AI.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan (klik judul untuk melompat)
1	**Mengenal React dan Next.js**	**Menjelaskan React sebagai pustaka komponen dan Next.js sebagai kerangka kerjanya**
2	JSX	Menulis tampilan dengan JSX dan memahami aturannya
3	Komponen, props, dan state	Membuat komponen yang menerima props dan menyimpan state
4	Siklus hidup komponen	Menjelaskan kapan komponen dipasang, diperbarui, dan dilepas
5	Hooks	Memakai hooks utama: useState, useEffect, useContext, useMemo, dan useRef
6	Pengambilan data	Mengambil data dari API dengan keadaan memuat dan error
7	Routing di Next.js	Membuat halaman, tautan, dan halaman dinamis dengan App Router
8	State management	Mengelola formulir dan berbagi state antarkomponen dengan React

Pustaka dan kerangka kerjanya

ITERA

React dan Next.js

React adalah pustaka JavaScript untuk membangun antarmuka dari komponen. **Next.js** adalah kerangka kerja di atas React yang menyediakan routing, rendering di server, dan alat build.

React sendiri hanya mengurus tampilan. Hal lain seperti halaman, URL, dan pengambilan data disediakan Next.js. Proyek mata kuliah ini memakai Next.js sebagai frontend yang memanggil API FastAPI.

```
npx create-next-app@latest krs-app
cd krs-app
npm run dev
# buka http://localhost:3000
```

Struktur proyek Next.js

ITERA

Berkas atau folder	Isinya
`app/page.js`	halaman /; inilah jawaban misi 1
`app/layout.js`	kerangka yang membungkus semua halaman: html, body, menu
`app/krs/page.js`	halaman `/krs`: nama folder menjadi path
`components/`	komponen yang dipakai ulang, misalnya kartu
`*.module.css`	CSS khusus satu komponen; nama kelas tidak bertabrakan
`public/`	berkas statis: gambar, JSON contoh
`package.json`	daftar pustaka dan perintah `dev`, `build`, `start`

Ikuti bersama: proyek React pertama

ITERA

1. Buka terminal di folder `starter/`, pasang pustaka.
   ```
   npm install
   ```
2. Jalankan server pengembangan.
   ```
   npm run dev
   ```
3. Buka alamatnya di browser.
   `http://localhost:3000`
4. Buka `app/page.js`, ganti teks judul dengan nama kalian, lalu simpan.
5. Lihat browser tanpa memuat ulang.

YANG HARUS TERLIHAT

Terminal menulis *Ready* dan alamat `localhost:3000`. Judul di browser berubah sendiri sesaat setelah berkas disimpan.

Kalau berbeda

npm tidak dikenal: pasang Node.js, buka terminal baru. *Port 3000 is in use*: buka port lain yang ditulis terminal. Judul tetap: simpan `app/page.js`.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Mengenal React dan Next.js	Inti: React membangun antarmuka dari komponen; Next.js menambah routing, rendering server, dan build.
2	**JSX**	**Menulis tampilan dengan JSX dan memahami aturannya**
3	Komponen, props, dan state	Membuat komponen yang menerima props dan menyimpan state
4	Siklus hidup komponen	Menjelaskan kapan komponen dipasang, diperbarui, dan dilepas
5	Hooks	Memakai hooks utama: useState, useEffect, useContext, useMemo, dan useRef
6	Pengambilan data	Mengambil data dari API dengan keadaan memuat dan error
7	Routing di Next.js	Membuat halaman, tautan, dan halaman dinamis dengan App Router
8	State management	Mengelola formulir dan berbagi state antarkomponen dengan React

Aturan JSX

```
export default function Profil() {
  const nama = "Rani";
  const sks = 20;
  return (
    <section className="kartu">
      <h2>{nama}</h2>
      <p>{sks ⩾ 24 ? "SKS penuh" : `${sks} SKS`}</p>
      <img src="/foto.jpg" alt={`Foto ${nama}`} />
      <label htmlFor="cari">Cari</label>
    </section>
  );
}
```

Elemen kosong wajib ditutup: `<img />`. Kembalikan satu induk, atau bungkus dengan `<> ... </>`.

Gaya dengan CSS Modules

ITERA

components/kartu.module.css

```
.kartu {
  border: 2px solid var(--utama);
  border-radius: 8px;
  padding: 12px 16px;
}
.kartu:hover { background: #F6EDCF; }
```

components/KartuMatkul.js

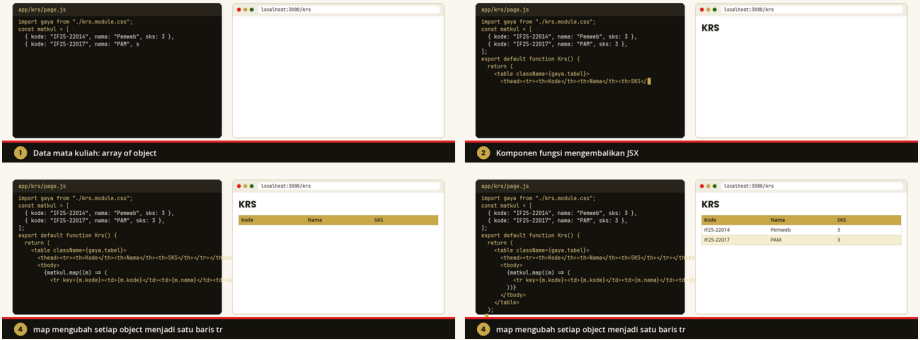
```
import gaya from "./kartu.module.css";

export default function KartuMatkul({ nama, sks
}) {
  return <article className={gaya.kartu}>
    <h3>{nama}</h3><p>{sks} SKS</p>
  </article>;
}
```

Bedanya

CSS dari P3 dan P4 dipakai apa adanya. Nama kelas diubah otomatis menjadi unik per komponen, jadi `.kartu` di dua komponen tidak saling menimpa. Gaya umum seperti variabel `--utama` ditulis di `app/globals.css`.

Dari data ke tabel



Satu komponen, satu array, satu map: tabel terbangun tanpa pustaka tambahan.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Mengenal React dan Next.js	Inti: React membangun antarmuka dari komponen; Next.js menambah routing, rendering server, dan build.
2	JSX	Inti: JSX mirip HTML di JavaScript: className, kurung kurawal untuk ekspresi, satu induk.
3	**Komponen, props, dan state**	**Membuat komponen yang menerima props dan menyimpan state**
4	Siklus hidup komponen	Menjelaskan kapan komponen dipasang, diperbarui, dan dilepas
5	Hooks	Memakai hooks utama: useState, useEffect, useContext, useMemo, dan useRef
6	Pengambilan data	Mengambil data dari API dengan keadaan memuat dan error
7	Routing di Next.js	Membuat halaman, tautan, dan halaman dinamis dengan App Router
8	State management	Mengelola formulir dan berbagi state antarkomponen dengan React

Blok pembangun antarmuka

ITERA

Komponen
Fungsi JavaScript berawalan huruf besar yang menerima **props** dan mengembalikan JSX; satu komponen bisa dipakai berkali-kali.

Inilah jawaban masalah minggu ini: kartu ditulis sekali sebagai komponen, lalu dipakai sepuluh kali dengan data berbeda lewat `map`. Setiap item daftar butuh `key` unik supaya React tahu mana yang berubah.

```
function KartuMatkul({ nama, sks }) {
  return <div className="kartu">
    <h3>{nama}</h3><p>{sks} SKS</p>
  </div>;
}
// di halaman:
{matkul.map((m) ⇒ (
  <KartuMatkul   key={m.kode}   nama={m.nama}
sks={m.sks} />
))}
```

State: data yang berubah

```
"use client";
import { useState } from "react";

export default function Krs({ matkul }) {
  const [diambil, setDiambil] = useState([]);
  const total = diambil.reduce((a, m) ⇒ a + m.sks, 0);

  const ambil = (m) ⇒ setDiambil([...diambil, m]);

  return (<>
    {matkul.map((m) ⇒ (
      <button        key={m.kode}        onClick={()        ⇒
ambil(m)}>{m.nama}</button>
    ))}
    <p>Total: {total} SKS</p>
  </>);
}
```

`onClick={() ⇒ ambil(m)}`: kirim fungsi, bukan memanggilnya. Kesalahan yang sama dengan P5.

Props atau state?

ITERA

Komponen `KartuMatkul` menampilkan nama mata kuliah dari induknya dan tanda centang yang berubah saat kartunya diklik. Mana yang props dan mana yang state?

- **A** keduanya props
- **B** nama props, centang state
- **C** nama state, centang props
- **D** keduanya state

JAWABAN: B

Nama datang dari luar dan tidak diubah kartu: props. Centang berubah karena tindakan di kartu itu sendiri: state.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Mengenal React dan Next.js	**Inti:** React membangun antarmuka dari komponen; Next.js menambah routing, rendering server, dan build.
2	JSX	**Inti:** JSX mirip HTML di JavaScript: className, kurung kurawal untuk ekspresi, satu induk.
3	Komponen, props, dan state	**Inti:** Komponen fungsi pengembali JSX; props dari induk dibaca saja, state milik komponen.
4	**Siklus hidup komponen**	**Menjelaskan kapan komponen dipasang, diperbarui, dan dilepas**
5	Hooks	Memakai hooks utama: useState, useEffect, useContext, useMemo, dan useRef
6	Pengambilan data	Mengambil data dari API dengan keadaan memuat dan error
7	Routing di Next.js	Membuat halaman, tautan, dan halaman dinamis dengan App Router
8	State management	Mengelola formulir dan berbagi state antarkomponen dengan React

Tiga tahap kehidupan

ITERA

Komponen fungsi menjalani tiga tahap. Kelas komponen lama punya method untuk tiap tahap; sekarang semuanya lewat `useEffect`.

1 Mounting

Pertama kali tampil. Tempat yang tepat untuk mengambil data awal.

2 Updating

Tampil ulang setiap props atau state berubah. Fungsi komponen dijalankan lagi dari atas.

3 Unmounting

Hilang dari halaman, misalnya pindah halaman. Bersihkan timer atau langganan di sini.

Ingat

Karena fungsi komponen dijalankan ulang setiap update, jangan memanggil `fetch` langsung di badan fungsi; pakai `useEffect`.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Mengenal React dan Next.js	**Inti:** React membangun antarmuka dari komponen; Next.js menambah routing, rendering server, dan build.
2	JSX	**Inti:** JSX mirip HTML di JavaScript: className, kurung kurawal untuk ekspresi, satu induk.
3	Komponen, props, dan state	**Inti:** Komponen fungsi pengembali JSX; props dari induk dibaca saja, state milik komponen.
4	Siklus hidup komponen	**Inti:** Dipasang, diperbarui saat props atau state berubah, lalu dilepas; efek lewat useEffect.
5	**Hooks**	**Memakai hooks utama: useState, useEffect, useContext, useMemo, dan useRef**
6	Pengambilan data	Mengambil data dari API dengan keadaan memuat dan error
7	Routing di Next.js	Membuat halaman, tautan, dan halaman dinamis dengan App Router
8	State management	Mengelola formulir dan berbagi state antarkomponen dengan React

useEffect

```
useEffect(() ⇒ {
  const id = setInterval(() ⇒ setDetik((d) ⇒ d + 1), 1000);
  return () ⇒ clearInterval(id); // saat dilepas
}, []); // [] = sekali saat dipasang

useEffect(() ⇒ {
  document.title = `KRS (${total} SKS)`;
}, [total]); // setiap total berubah
```

Daftar `[total]`: jalan lagi setiap `total` berubah. Lupa daftar sama sekali berarti jalan setiap tampil, sumber perulangan tanpa akhir bila efeknya mengubah state.

Hooks lain yang perlu dikenal

ITERA

Hook	Gunanya	Contoh
`useContext`	berbagi data ke banyak komponen tanpa diteruskan lewat props berlapis	tema gelap, data pengguna yang login
`useMemo`	mengingat hasil hitungan berat sampai masukannya berubah	menyaring 2000 baris nilai
`useRef`	menyimpan nilai atau elemen DOM tanpa menggambar ulang	memfokuskan kotak isian
`useReducer`	state rumit dengan banyak aksi	keranjang KRS dengan tambah, hapus, kosongkan

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Mengenal React dan Next.js	**Inti:** React membangun antarmuka dari komponen; Next.js menambah routing, rendering server, dan build.
2	JSX	**Inti:** JSX mirip HTML di JavaScript: className, kurung kurawal untuk ekspresi, satu induk.
3	Komponen, props, dan state	**Inti:** Komponen fungsi pengembali JSX; props dari induk dibaca saja, state milik komponen.
4	Siklus hidup komponen	**Inti:** Dipasang, diperbarui saat props atau state berubah, lalu dilepas; efek lewat useEffect.
5	Hooks	**Inti:** useState menyimpan, useEffect menjalankan efek, useContext berbagi data.
6	**Pengambilan data**	**Mengambil data dari API dengan keadaan memuat dan error**
7	Routing di Next.js	Membuat halaman, tautan, dan halaman dinamis dengan App Router
8	State management	Mengelola formulir dan berbagi state antarkomponen dengan React

fetch di komponen klien

```
"use client";
export default function DaftarMatkul() {
  const [data, setData] = useState([]);
  const [memuat, setMemuat] = useState(true);
  const [galat, setGalat] = useState(null);
  useEffect(() => {
    fetch("http://localhost:8000/matkul")
      .then((r) => r.ok ? r.json() : Promise.reject(r.status))
      .then(setData).catch(setGalat)
      .finally(() => setMemuat(false));
  }, []);
  if (memuat) return <p>Memuat mata kuliah...</p>;
  if (galat) return <p role="alert">Gagal memuat. Coba lagi.</p>;
  return            <ul>{data.map((m)           =>           <li
key={m.kode}>{m.nama}</li>)}</ul>;
}
```

Komponen server di Next.js bisa langsung `await fetch(...)` tanpa state. Pustaka seperti SWR atau TanStack Query menambah cache dan muat ulang otomatis.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Mengenal React dan Next.js	**Inti:** React membangun antarmuka dari komponen; Next.js menambah routing, rendering server, dan build.
2	JSX	**Inti:** JSX mirip HTML di JavaScript: className, kurung kurawal untuk ekspresi, satu induk.
3	Komponen, props, dan state	**Inti:** Komponen fungsi pengembali JSX; props dari induk dibaca saja, state milik komponen.
4	Siklus hidup komponen	**Inti:** Dipasang, diperbarui saat props atau state berubah, lalu dilepas; efek lewat useEffect.
5	Hooks	**Inti:** useState menyimpan, useEffect menjalankan efek, useContext berbagi data.
6	Pengambilan data	**Inti:** fetch di useEffect dengan state data, memuat, dan galat; komponen server boleh await.
7	**Routing di Next.js**	**Membuat halaman, tautan, dan halaman dinamis dengan App Router**
8	State management	Mengelola formulir dan berbagi state antarkomponen dengan React

Folder menjadi URL

ITERA

Berkas	URL
`app/page.js`	`/`
`app/tentang/page.js`	`/tentang`: jawaban misi 2
`app/matkul/page.js`	`/matkul`
`app/matkul/[kode]/page.js`	`/matkul/IF25-22014`, `/matkul/IF25-22017`, dan seterusnya
`app/layout.js`	membungkus semua halaman di bawahnya

Link dan halaman dinamis

ITERA

app/matkul/page.js

```
import Link from "next/link";

<Link href={`/matkul/${m.kode}`}>
  {m.nama}
</Link>
```

app/matkul/[kode]/page.js

```
"use client";
import { useParams, useRouter }
  from "next/navigation";

export default function Detail() {
  const { kode } = useParams();
  const router = useRouter();
  return <button onClick={() ⇒
    router.push("/matkul")}>{kode}</button>;
}
```

Bedanya

`Link` berpindah tanpa memuat ulang; `[kode]` menangkap bagian URL; `useParams` membacanya.

Delapan bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Mengenal React dan Next.js	**Inti:** React membangun antarmuka dari komponen; Next.js menambah routing, rendering server, dan build.
2	JSX	**Inti:** JSX mirip HTML di JavaScript: className, kurung kurawal untuk ekspresi, satu induk.
3	Komponen, props, dan state	**Inti:** Komponen fungsi pengembali JSX; props dari induk dibaca saja, state milik komponen.
4	Siklus hidup komponen	**Inti:** Dipasang, diperbarui saat props atau state berubah, lalu dilepas; efek lewat useEffect.
5	Hooks	**Inti:** useState menyimpan, useEffect menjalankan efek, useContext berbagi data.
6	Pengambilan data	**Inti:** fetch di useEffect dengan state data, memuat, dan galat; komponen server boleh await.
7	Routing di Next.js	**Inti:** Folder menjadi path, page.js halamannya, [kode] halaman dinamis, Link berpindah.
8	**State management**	**Mengelola formulir dan berbagi state antarkomponen dengan React**

Formulir terkendali

```
export default function Tambah({ onTambah }) {
  const [nama, setNama] = useState("");
  const [galat, setGalat] = useState("");
  function kirim(e) {
    e.preventDefault();
    if (nama.trim().length < 3) return setGalat(
      "Nama minimal 3 huruf.");
    onTambah({ nama }); setNama("");
  }
  return <form onSubmit={kirim}>
    <label htmlFor="nama">Nama</label> <input id="nama"
      value={nama} onChange={(e) => {
        setNama(e.target.value); setGalat(""); }} />
    {galat && <p role="alert">{galat}</p>}
    <button type="submit">Tambah mata kuliah</button>
  </form>;
}
```

Pola yang sama dipakai untuk menampilkan pesan 422 dari FastAPI di P11.

Context dan useReducer

```
"use client"; // app/KrsProvider.js
const KrsContext = createContext(null);
function reducer(state, aksi) { switch (aksi.jenis) {
    case "ambil": return [...state, aksi.matkul];
    case "lepas":
      return state.filter((m) => m.kode !== aksi.kode);
    default: return state;
}}
export function KrsProvider({ children }) {
  const [diambil, kirim] = useReducer(reducer, []);
  return <KrsContext.Provider value={{ diambil, kirim }}>
    {children}
  </KrsContext.Provider>;
}
export const useKrs = () => useContext(KrsContext);
```

Untuk aplikasi besar, pustaka seperti Redux atau Zustand memakai gagasan yang sama. State management ini dari slide lama.

Tiga baris yang salah

ITERA

Yang ditulis	Mengapa salah, dan perbaikannya
`useState` di `page.js` tanpa `"use client"`	Komponen di folder app adalah komponen server secara bawaan; hooks tidak tersedia. Tambahkan `"use client"` di baris pertama.
`{matkul.map((m) ⇒ <Kartu nama={m.nama} />)}`	Tanpa `key`; React memberi peringatan dan bisa salah memperbarui baris. Tambahkan `key={m.kode}`.
`diambil.push(m); setDiambil(diambil);`	Array yang sama diubah langsung, React tidak melihat perubahan. Pakai `setDiambil([...diambil, m])`.

Pola kesalahannya

Dua yang terakhir tidak memunculkan error merah, hanya tampilan yang tidak berubah atau peringatan di Console.

Giliran kalian

ITERA

1 **Setengah jadi: penghitung SKS**

Lengkapi: `const [total, _____] = useState(0);` dan tombol `<button onClick={_____}>Tambah 3 SKS</button>`.

2 **Mandiri: halaman detail**

Tulis `app/dosen/[nip]/page.js` yang menampilkan NIP dari URL di dalam `<article>` bergaya CSS Modules, dan tombol *Kembali ke daftar dosen*.

JAWABAN

Setengah jadi: `setTotal`, `() ⇒ setTotal(total + 3)`. Mandiri: ”use client”, useParams untuk nip, useRouter untuk kembali, import gaya dari berkas module.css.

Hands-on bertingkat

Tingkat	Waktu	Yang dilatih	Sub-CPMK
Tingkat 1, Dasar	10′	komponen dengan props, CSS Modules, dan `map` dengan `key`	7.1
Tingkat 2, Menengah	15′	state total SKS dan pengambilan data dengan keadaan memuat dan gagal	7.1, 7.2
Tingkat 3, Tantangan	10′	halaman dinamis, Link, tabel, dan formulir terkendali	7.2
Pengayaan	sisa	Context untuk total SKS di menu atas	7.2

Aturan

Berkas awal ada di `02 Hands-on/P7 - React JS - Hands-on/starter`. Kerjakan berurutan dari tingkat 1, simpan bukti tangkapan layar, dan sebutkan alasan setiap langkah.

Presentasi di papan

ITERA

Tiga mahasiswa diundi untuk menunjukkan hasil hands-on tingkat 2 atau 3 di layar, masing-masing sekitar 5 menit, lalu menjawab satu pertanyaan.

Kriteria	Sangat baik 75--100	Baik 65--74	Cukup 55--64	Kurang <55
Ketepatan hasil	berjalan dan sesuai soal	satu salah kecil	ada bagian salah	sebagian besar keliru
Alasan tiap langkah	setiap baris dijelaskan fungsinya	sebagian besar dijelaskan	penjelasan kabur	tanpa penjelasan
Menjawab pertanyaan	tanpa melihat catatan	dengan sedikit bantuan	menjawab sebagian	tidak bisa menjawab

Yang tidak maju tetap bisa menambah nilai lewat pertanyaan atau koreksi yang tepat.

Exit ticket Pertemuan 7

ITERA

```
function Kartu({ nama }) { return <h3>{nama}</h3>; }
const [sks, setSks] = useState(0);
```

1 Sub-CPMK 7.1

Di komponen `Kartu`, apa itu `nama`: props atau state? Mengapa kartu tidak boleh mengubah nilainya sendiri?

2 Sub-CPMK 7.2

Tulis berkas dan isi minimal agar halaman `/profil` menampilkan tombol yang menambah `sks` sebanyak *A* setiap diklik.

3 Refleksi

Tulis satu kalimat: bagian mana yang paling membingungkan hari ini, dan mengapa?

Soal 1 dan 2 dinilai 0, 1, atau 2: benar dengan alasan, benar tanpa alasan atau salah kecil, belum benar.

Yang dibawa pulang

ITERA

	Bagian	Inti
1	Mengenal React dan Next.js	React membangun antarmuka dari komponen; Next.js menambah routing, rendering server, dan build.
2	JSX	JSX mirip HTML di JavaScript: className, kurung kurawal untuk ekspresi, satu induk.
3	Komponen, props, dan state	Komponen fungsi pengembali JSX; props dari induk dibaca saja, state milik komponen.
4	Siklus hidup komponen	Dipasang, diperbarui saat props atau state berubah, lalu dilepas; efek lewat useEffect.
5	Hooks	useState menyimpan, useEffect menjalankan efek, useContext berbagi data.
6	Pengambilan data	fetch di useEffect dengan state data, memuat, dan galat; komponen server boleh await.
7	Routing di Next.js	Folder menjadi path, page.js halamannya, [kode] halaman dinamis, Link berpindah.
8	State management	Formulir terkendali menyimpan isian di state; Context dan useReducer membagikan state ke banyak komponen tanpa props berlapis.

Ada pertanyaan?

Sampai jumpa di Pertemuan 8: Ujian Tengah Semester

ITERA